Swinburne University of Technology
School of Science, Computing, and Emerging Technologies

SWE40006 - Software Deployment and Evolution

Deployment Portfolio Task 2

Student name: Ngoc Van Nhi Do
Student ID: 105551765
Submission date: 28/08/2026

Deployment Portfolio Task 2

1 Declared Target Level

All four subtasks 2.1, 2.2, 2.3, and 2.4 were attempted.

Table 1 below contain relevant links as proof of successful task execution.

Table 1: Publicly accessible URLs for live grading verification

Source	**URL**
Task 2.1	https://swe40006-task-2-1-atbmg9a7daapdnec.australiaeast-01.azurewebsites.net
Task 2.2	https://swe40006-task-2-2-f9emhph2gddzhker.australiaeast-01.azurewebsites.net/ (stopped)
Task 2.3	https://swe40006-task-2-3-hacthcdka0dub4ax.australiaeast-01.azurewebsites.net/ (stopped)
Task 2.4	https://swe40006-task-2-4-b2gacqbtckemadfp.australiaeast-01.azurewebsites.net/ (stopped)
Source code files	https://github.com/nhydnv/SWE40006-Software-Deployment-and-Evolution/tree/main/deployment_task_2

2 Execution Evidence

2.1 Task 2.1

For Task 2.1, I deployed a simple Node.js web application to Azure App Services, using Visual Studio Code and the Azure App Services extension.

First, I created the Node.js app consisting of the files `index.html`, `style.css`, `package.json` and `server.js`.

Figure 1: Structure of my simple Node.js application.

Then I ran my web application to test that it works correctly.

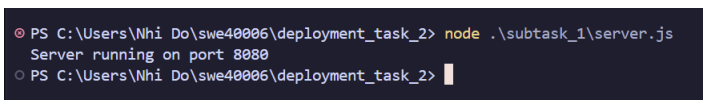


Figure 2: Running `npm start` on my Node.js application.

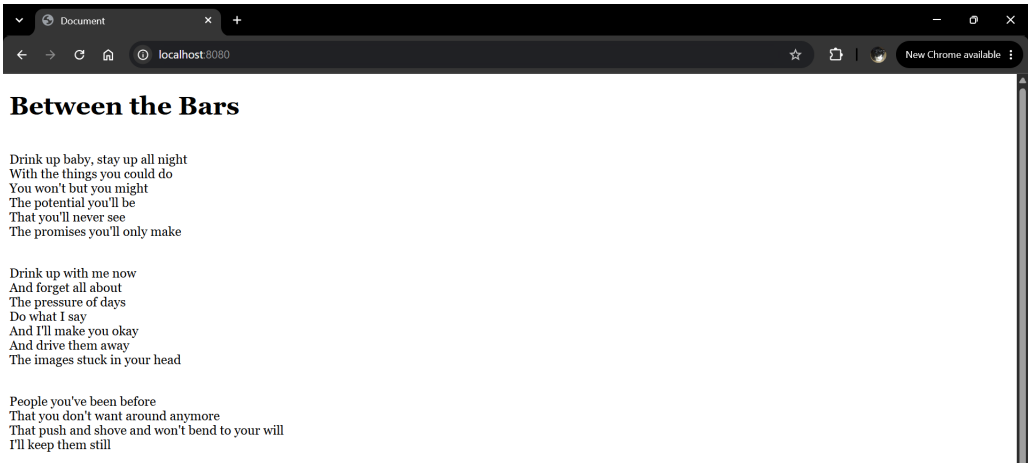


Figure 3: The web application runs locally on port 8080.

I then installed the Azure App Service on VS Code. The screenshot below shows the Azure panel once the extension has been successfully installed and I have logged in with my Azure account.

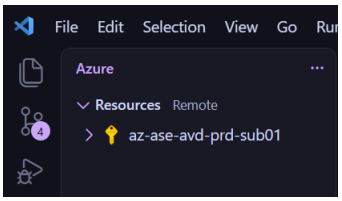


Figure 4: The Azure side panel on VS Code.

Next, I subscribed to an Azure for Students subscription on the Azure portal (portal.azure.com).

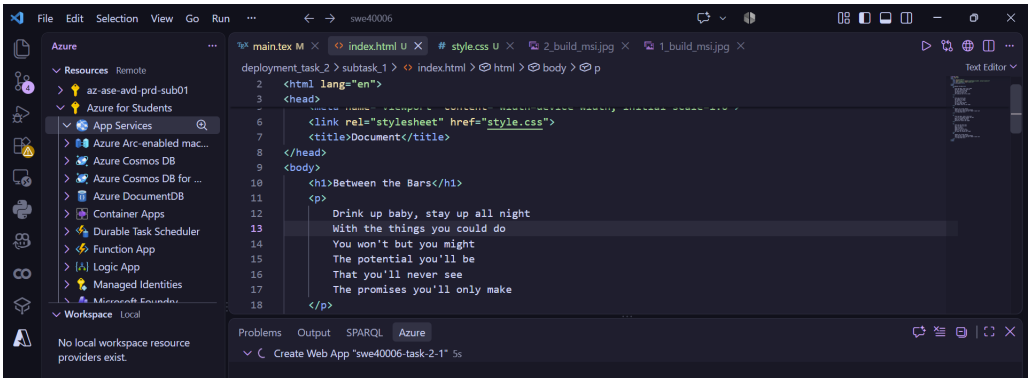


Figure 5: The Azure panel updated with my new Azure for Students subscription.

I then created a new Azure Web App service using the "Create Web App" command. I specified the region where I wanted my web app to be hosted which was Australia East, the application name `swe40006-task-2-1`, and the technology stack which was Node LTS 24.

As seen below, initial attempts to create the web app failed as my subscription was not registered with the `Microsoft.OperationalInsights` namespace. This was resolved by registering with the namespace on the Azure Portal.

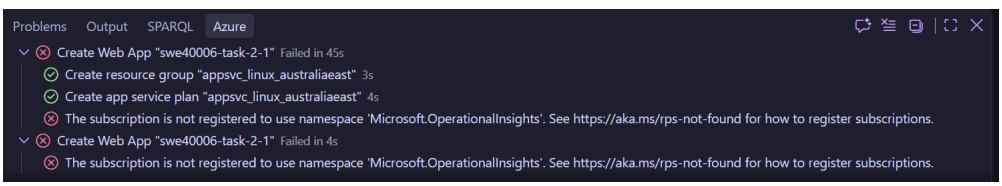


Figure 6: Failed attempts at creating a new Azure App Service instance.

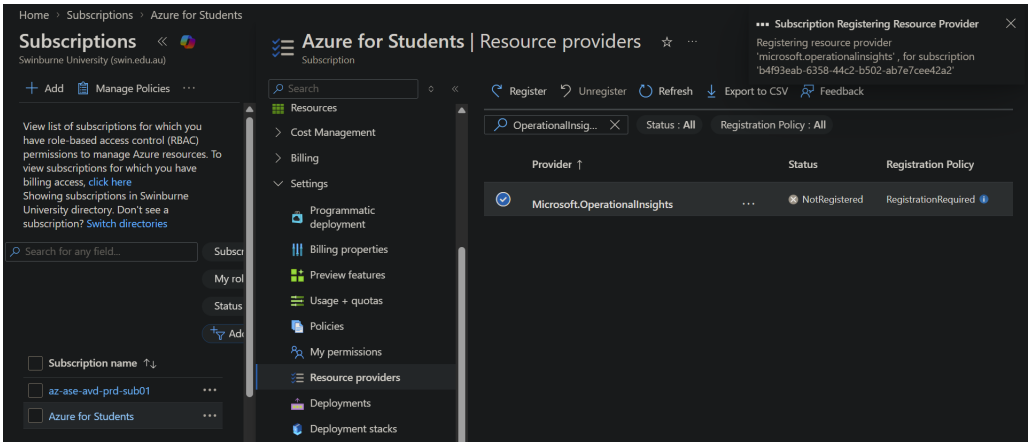


Figure 7: Registering with the `Microsoft.OperationalInsights` namespace on the Azure Portal.

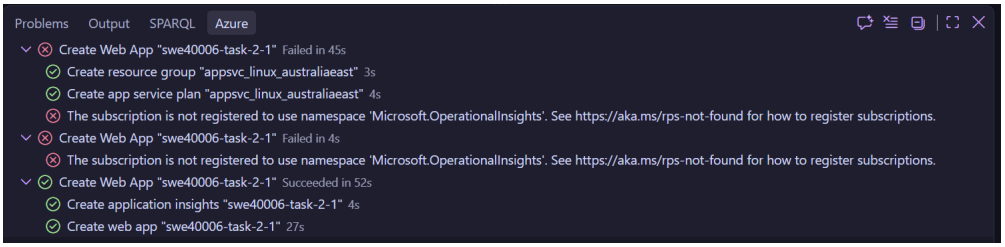


Figure 8: A new Azure App Service instance is successfully created.

The next step was to deploy my web application to the App Service instance.

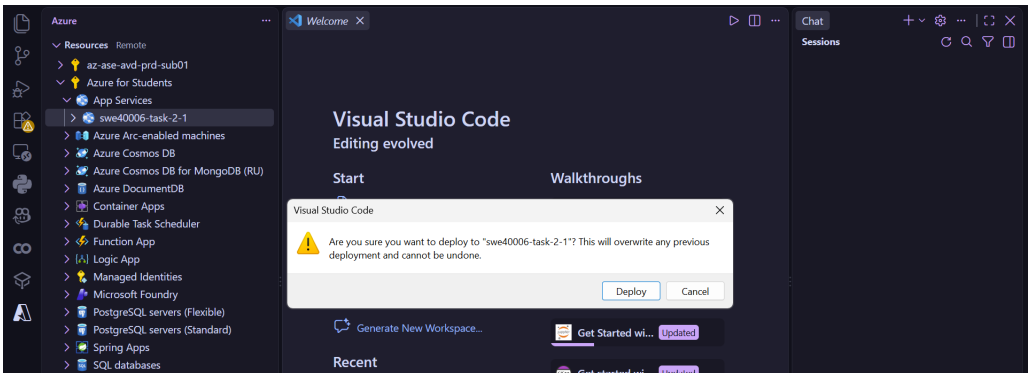


Figure 9: Deploying my web application to `swe40006-task-2-1`.

Figure 10: Deployment successful.

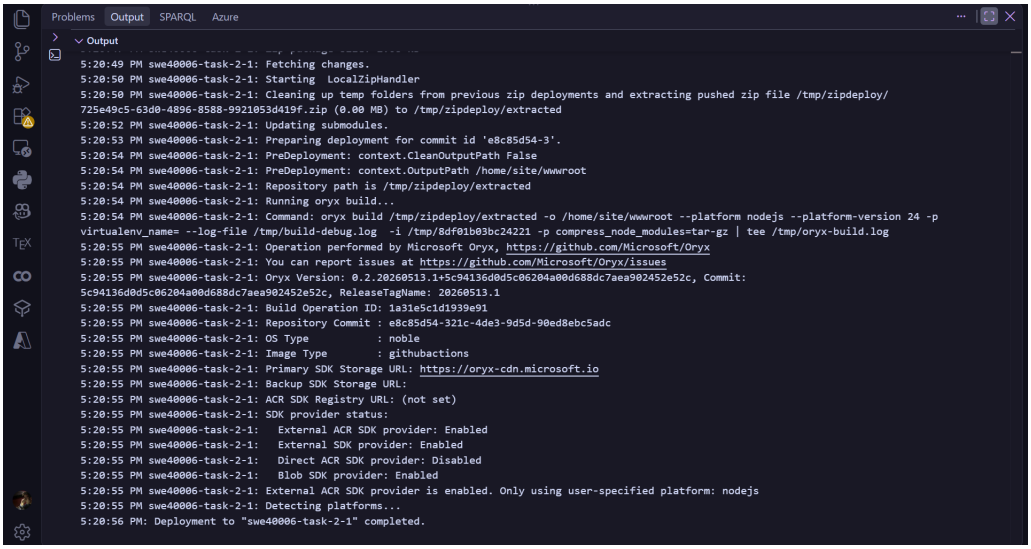


Figure 11: Build output.

The application is live and publicly acccessible at https://swe40006-task-2-1-atbmg9a7daapdnec.australiaeast-01.azurewebsites.net.

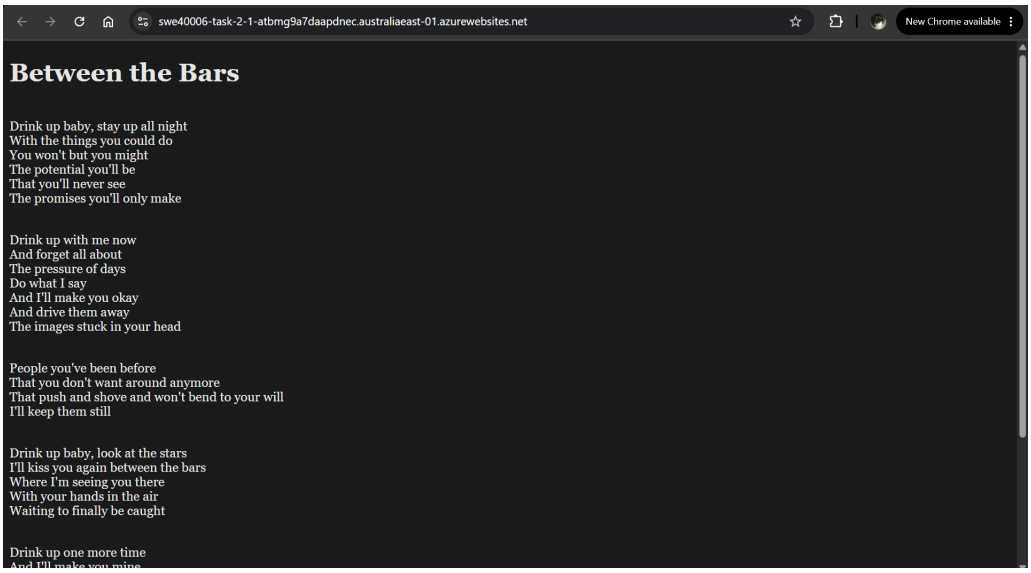


Figure 12: The application is deployed and running live.

2.2 Task 2.2

For task 2.2, I deployed an ASP.NET Razor Pages application with Azure App Services, also using Visual Studio Code and the Azure App Service extension.

First, I created a new Razor Pages project using the .NET CLI. Once the project has been created, I ran `dotnet publish` to publish the application.

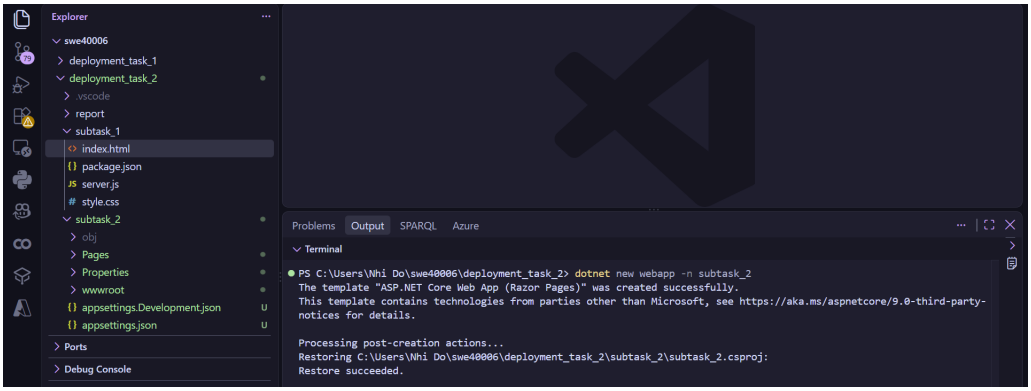


Figure 13: Creating a new Razor Pages project via the CLI.

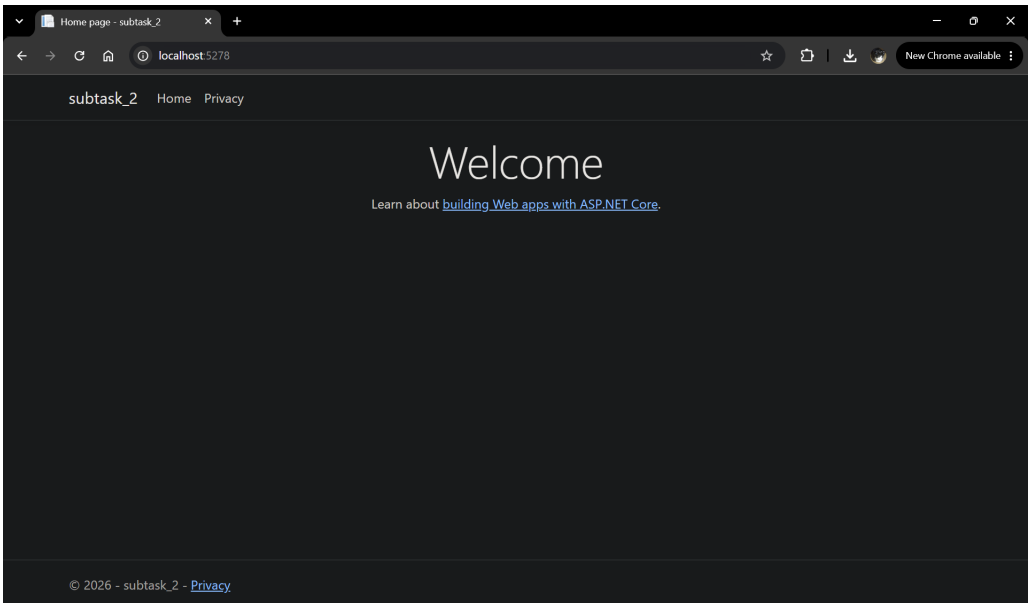


Figure 14: The application runs locally on port 5278.

Similar to Task 2.1, I created a new Web App Service instance using the Azure App Service extension. The new instance is named `swe40006-task-2-2`, hosted in the Australia East region, and using the .NET 8.0 runtime.

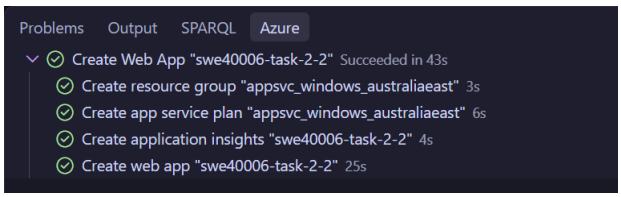


Figure 15: A new Web App Service instance is successfully created.

The application is then deployed into this new instance. It is hosted at https://swe40006-task-2-2-f9emhph2gddzhker.australiaeast-01.azurewebsites.net/.

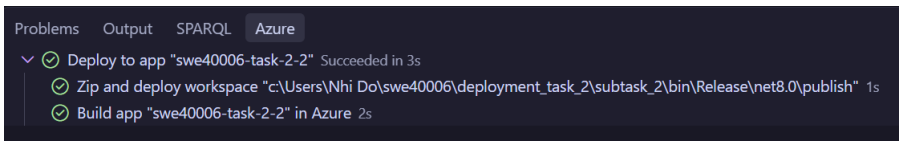


Figure 16: Successful deployment.

Figure 17: Logs showing that deployment was successful.

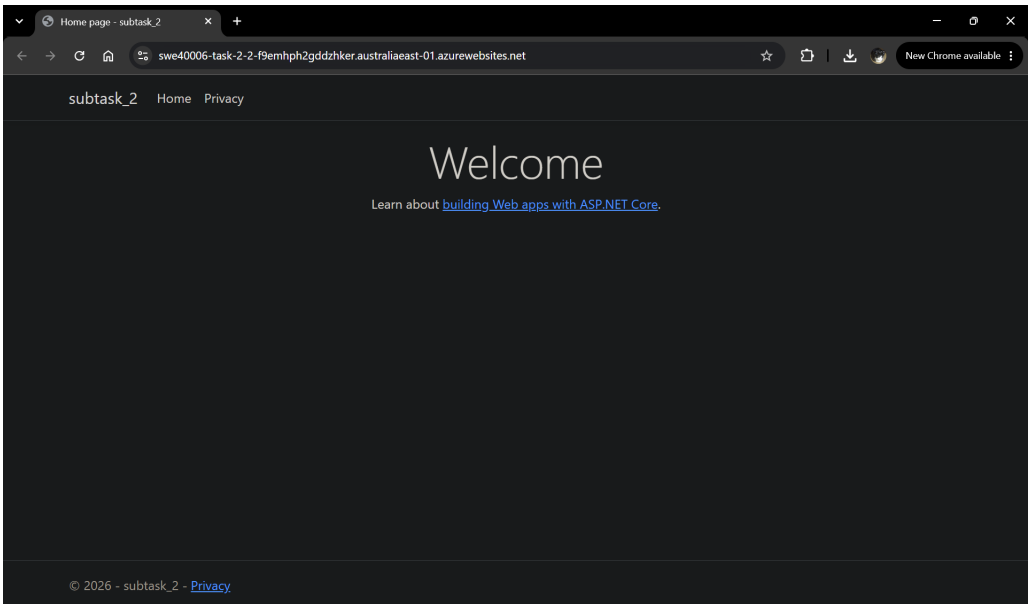


Figure 18: The web application up and running live.

Even though this App Service instance is running on a free tier, it is still best practice to manage cloud resources responsibly to avoid unintended charges. To demonstrate this, I stopped the instance using the Azure Portal.

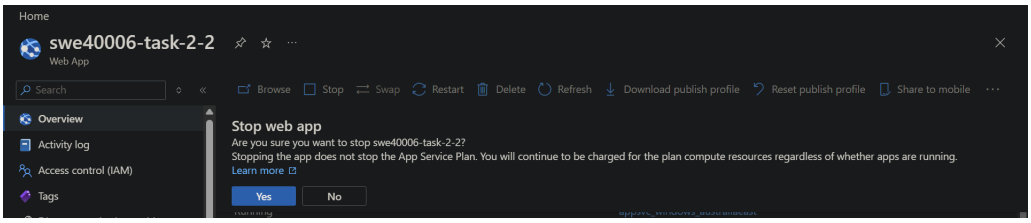


Figure 19: Stopping the instance.

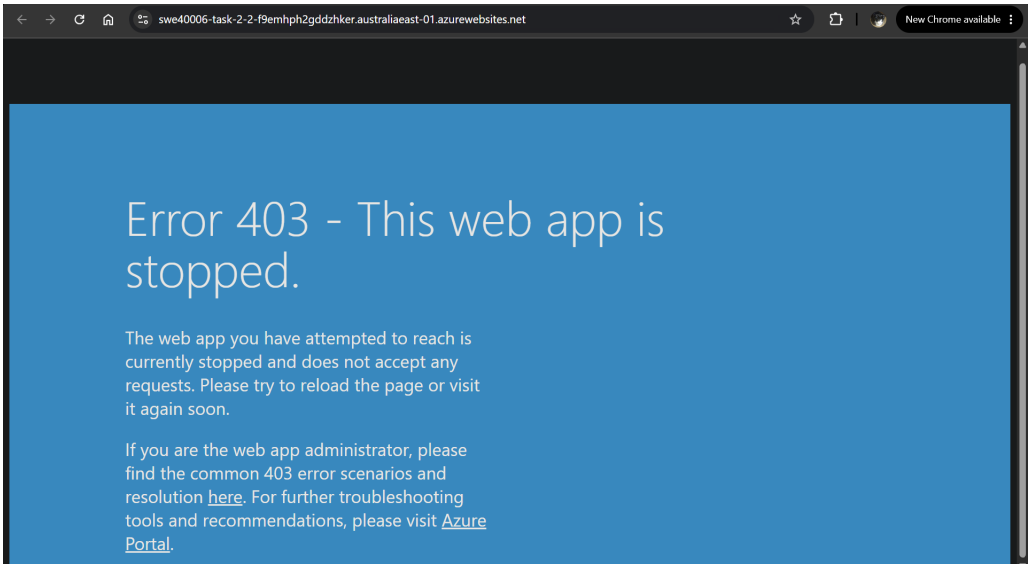


Figure 20: The web app has been stopped.

2.3 Task 2.3

For task 2.3, I deployed a copy of the Razor Pages application created in the previous task.

2.3.1 Configuration Management

To demonstrate configuration abstraction, I defined a setting in `appsettings.json` and read its value at runtime via `IConfiguration`, rather than hardcoding it directly in the application code. This value was then overriden using an Azure App Service Environment Variable in the Azure Portal, allowing the deployed application to resolve a different value. In a real scenario involving sensitive data, the value in `appsettings.json` would be left blank, and the real value stored locally using `user-secrets`. As the configuration value here is non-sensitive and only for demonstration purposes, I kept it in `appsettings.json` for easy testing.

First, I added the new setting `WelcomeMessage` in `appsettings.json`.

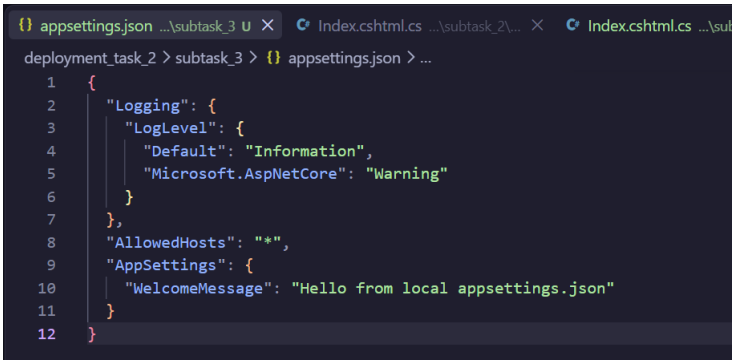


Figure 21: Adding a new setting in `appsettings.json`.

I then tested the application locally. The configuration value showed up as expected.

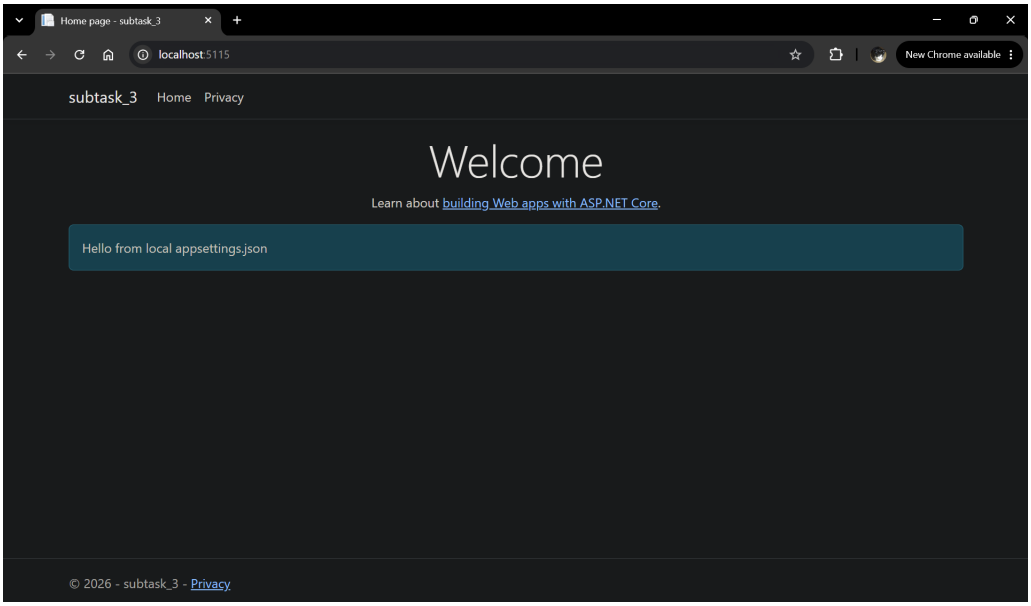


Figure 22: Testing that the app runs fine locally.

I then deployed the app with the Azure App Service extension similar to what I did in task 2.2.

Figure 23: Creating a new Web Service instance and deploying my app.

In the Azure Portal, I added a new environment variable to the newly created `swe40006-task-2-3` App Service. The value here is different from the value that was used locally ("Hello from Azure Web Service" vs. "Hello from local appsettings.json") to demonstrate the overriding mechanism.

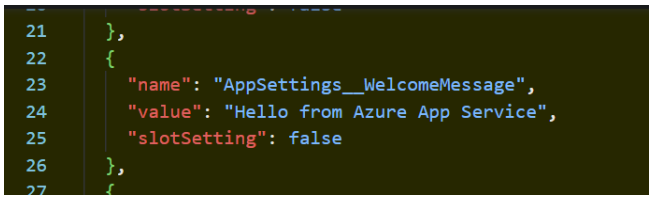


Figure 24: Adding a new environment variable `AppSettings__WelcomeMessage` in the Azure Portal.

The overriden value set to the environment variable showed up on the deployed website.

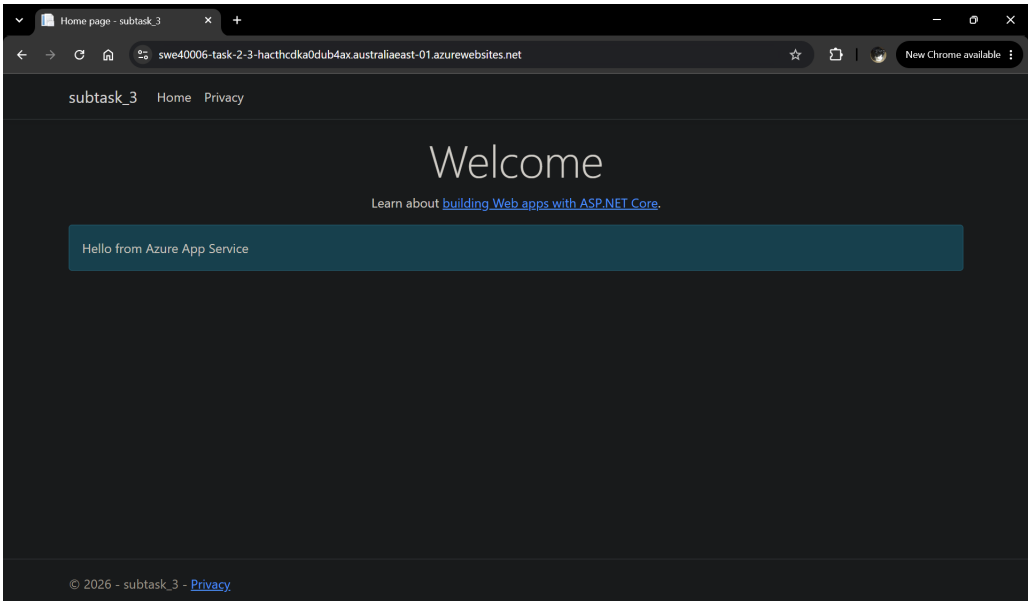


Figure 25: The site showing the new value.

2.3.2 Diagnostics and Logging

I also configured a active health check on this `swe40006-task-2-3` instance. First, I modified the source code to the the health check endpoint.

```
26  app.MapHealthChecks("/health");
27  app.Run();
```

```
3  // Add services to the container.
4  builder.Services.AddRazorPages();
5  builder.Services.AddHealthChecks();
6
```

Figure 26: Adding a new health check endpoint to the app.

Then, once on the Azure Portal, I realised that the health check wasn't available on the free tier. This was resolved by scaling the instance to a Basic plan.

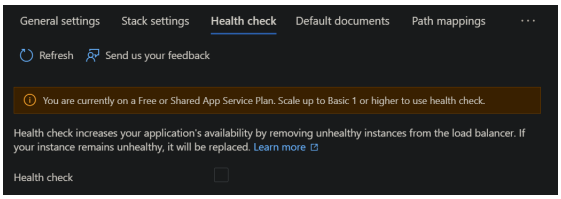


Figure 27: The health check is not available for free-tier instances.

Once that was resolved, I could configure the health check on the portal by adding the path to the health check endpoint, which was `/health`.

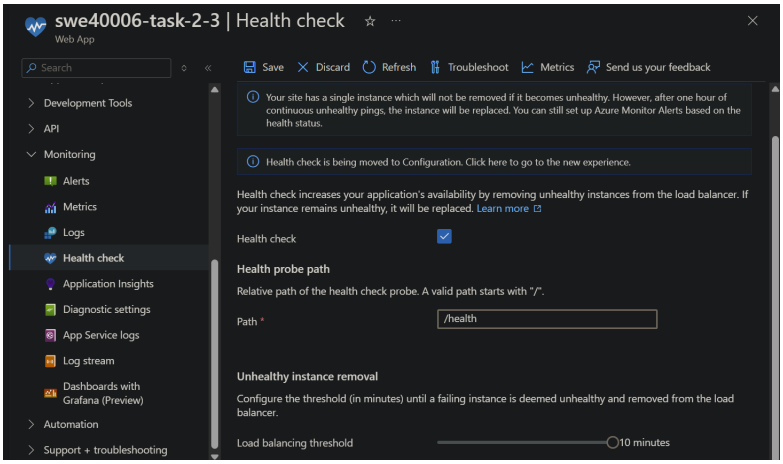


Figure 28: Setting up the active health check endpoint on Azure Portal.

Going to the endpoint at https://swe40006-task-2-3-hacthcdka0dub4ax.australiaeast-01.azurewebsites.net/health verified that the intstance was healthy. I then stopped the instance to ensure that resources weren't being wasted.

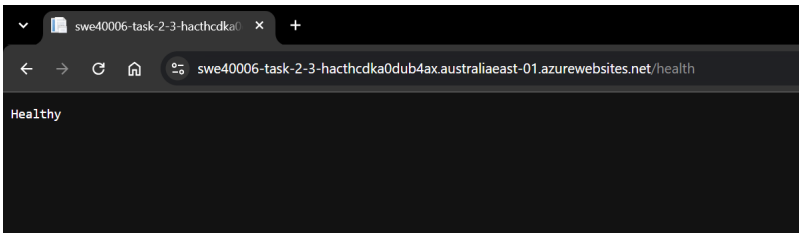


Figure 29: Verifying that active health checking is working.

I also configured Azure App Service Logging on my instance using the Azure Portal by enabling Application and Web Server logs.

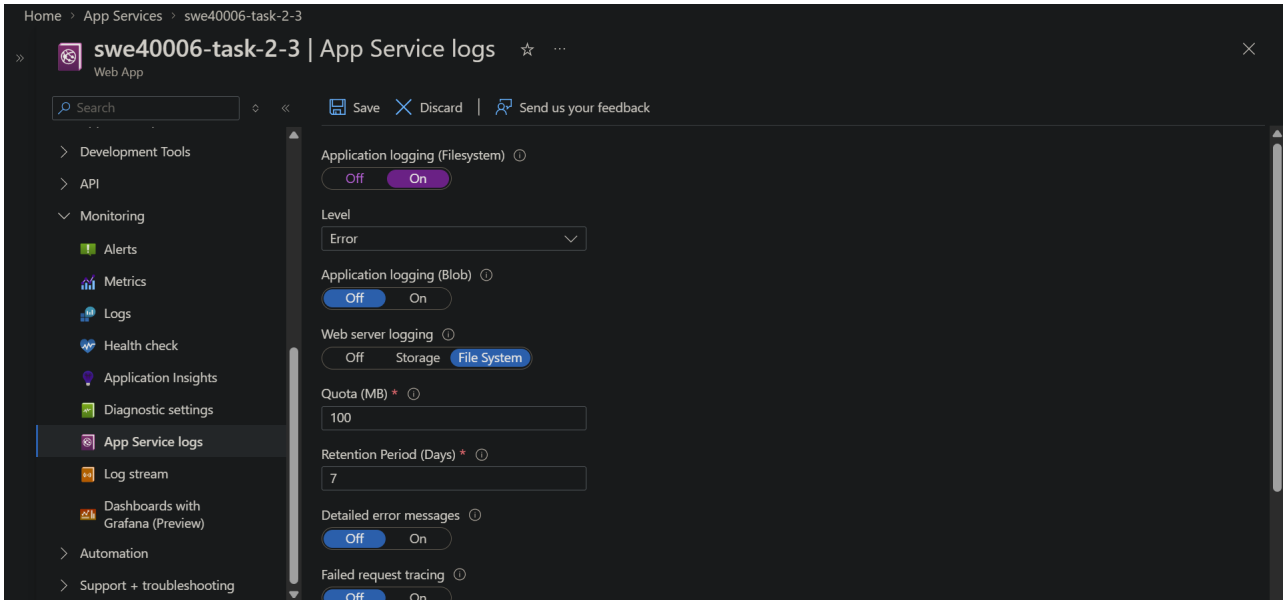


Figure 30: Configuring App Service logs on my instance.

The screenshots below show Azure App Service Log Stream capturing traffic in real time.

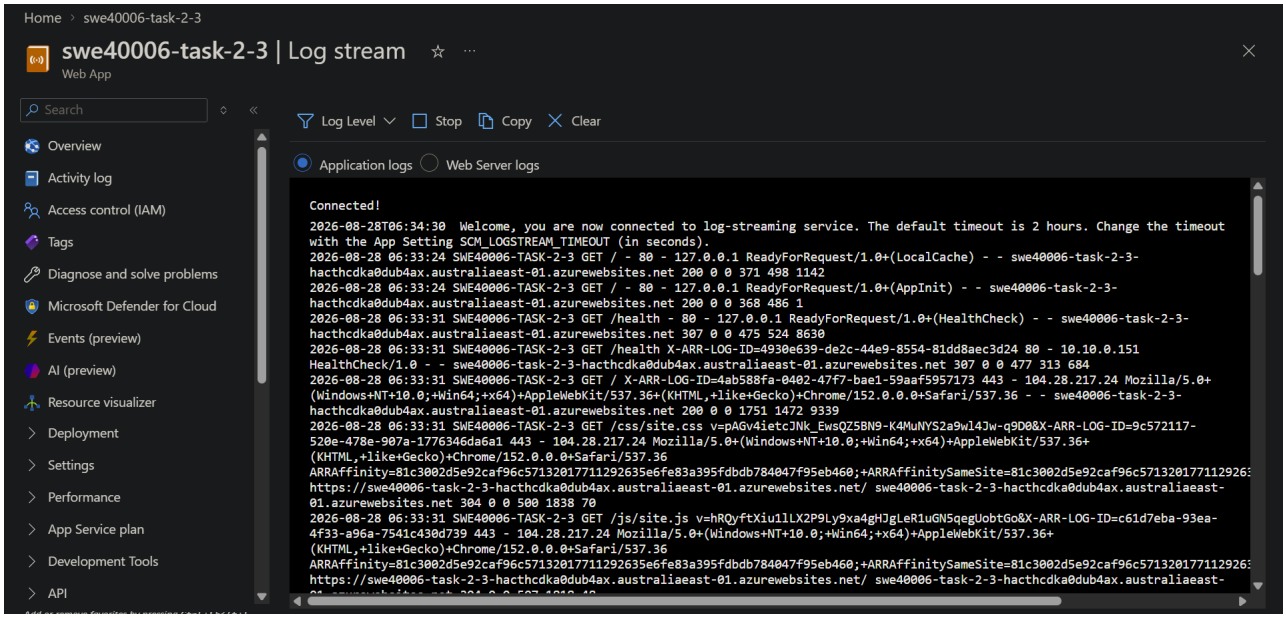


Figure 31: Application logs.

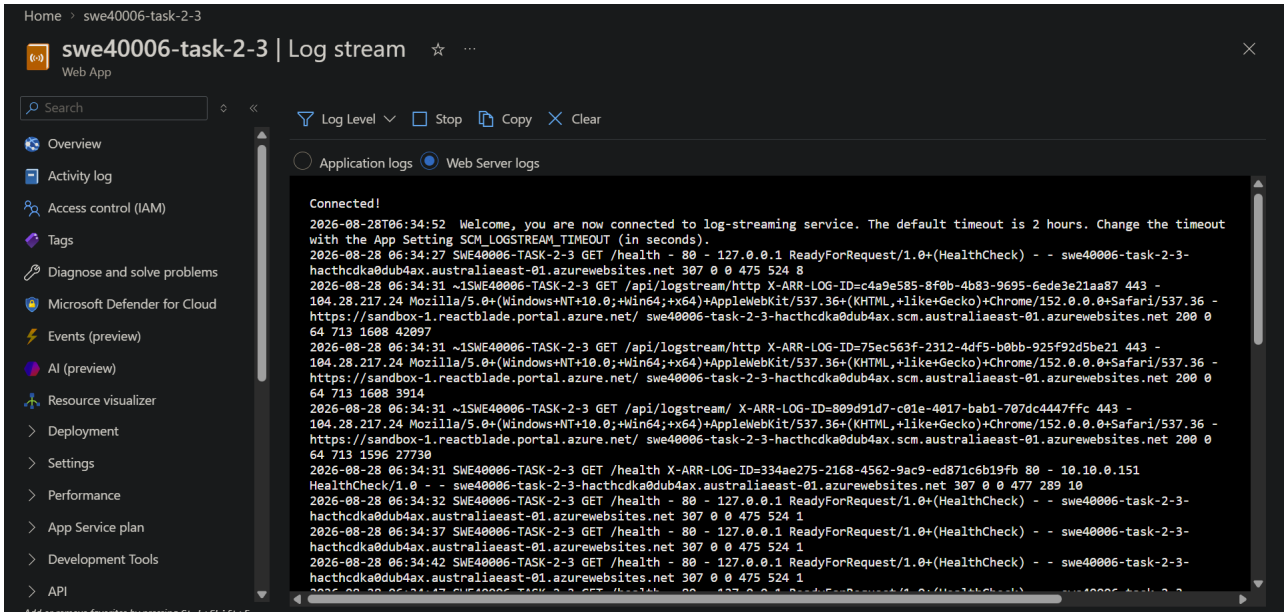


Figure 32: Web server logs.

2.4 Task 2.4

In task 2.4, I deployed a PHP web application to Azure App Service and integreated Azure Application Insights over one of my previously hosted apps.

First, I created a simple web application with two PHP files and a stylesheet.

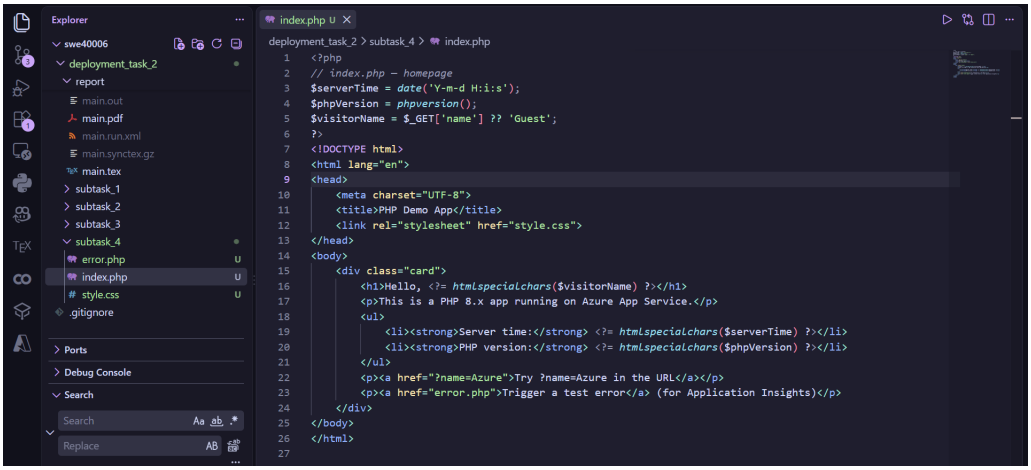


Figure 33: Creating a new PHP web application.

Once the application had been written, I deployed it in the same way I did in previous tasks. The deployed app could be found at https://swe40006-task-2-4-b2gacqbtckemadfp.australiaeast-01.azurewebsites.net/.

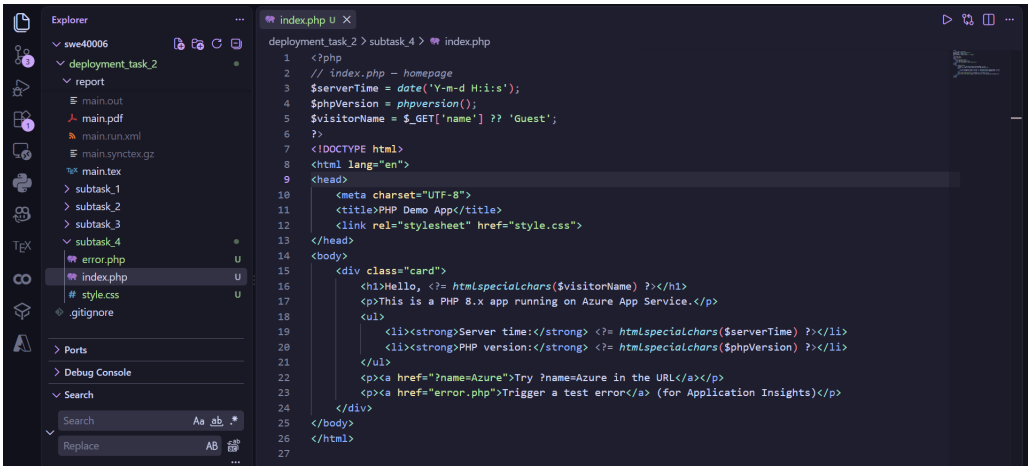


Figure 34: The deployed application running on Azure App Service.

Since there is no official server-side monitoring for PHP sites by Azure, I decided to integrate Application Insights for the ASP.NET application deployed in task 2.3. An Application Insights resource had been automatically created at deployment time for the app, as shown below:

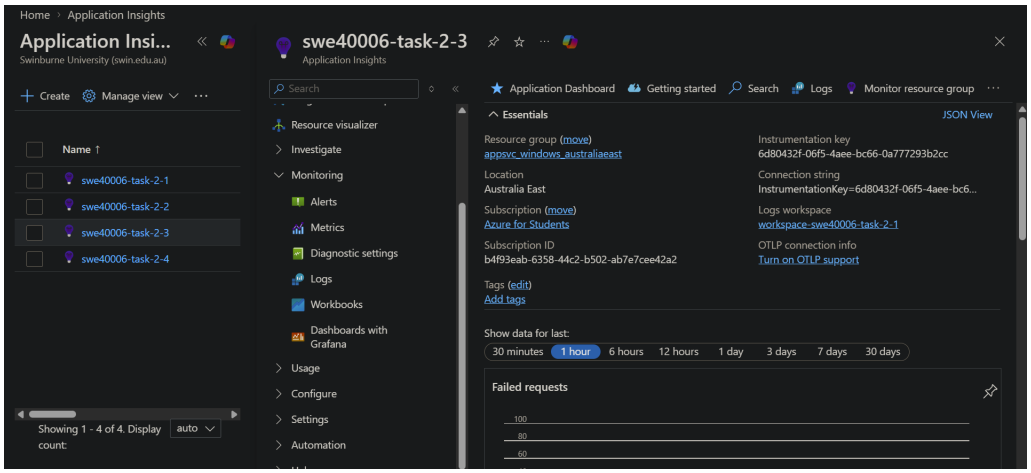
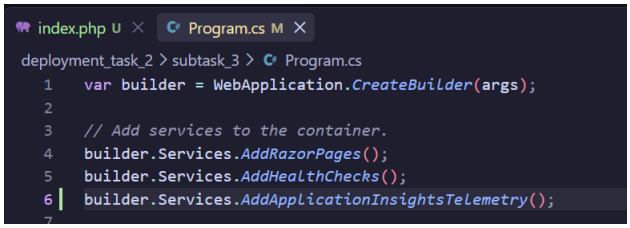


Figure 35: The Application Insights resource automatically created for the `swe40006-task-2-3` instance.

I then registered and configured the Application Insights services so that my app could send telemetry to Azure Application Insights by adding a line to the source code. I also wired the App Service instance to the Application Insights resource mentioned above on the Azure Portal.



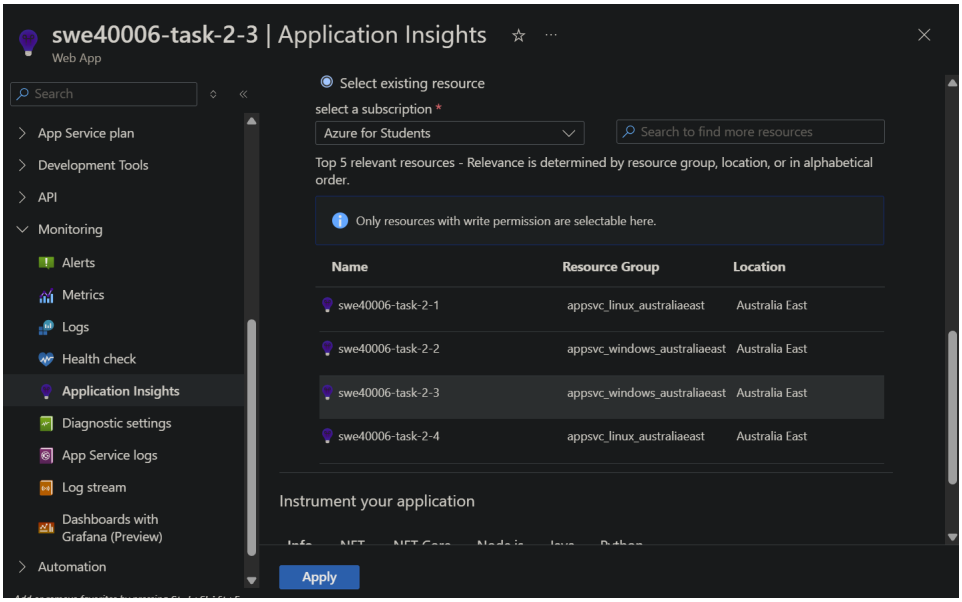


Figure 36: Configuring Application Insights for my app.

After those had been configured, I could track the application's live metrics, performance metrics, and failures on the portal.

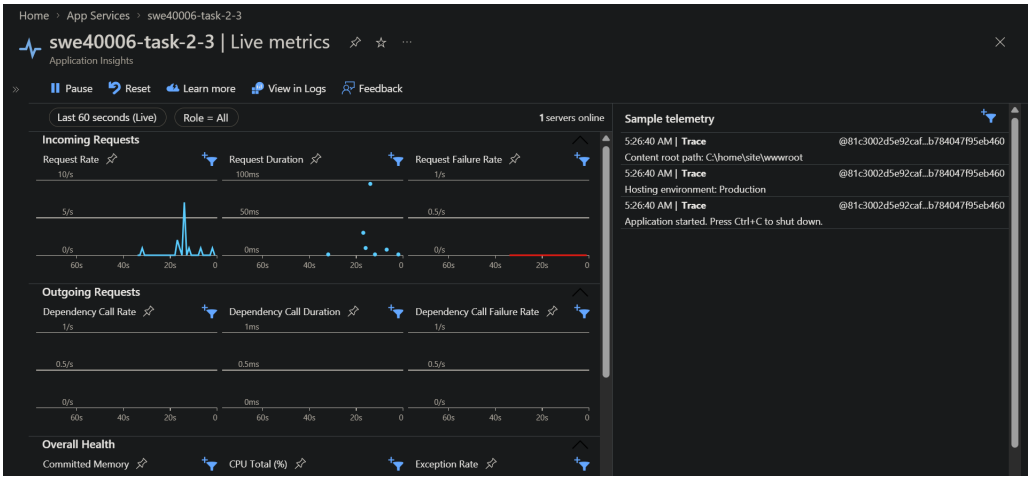


Figure 37: Live metrics.

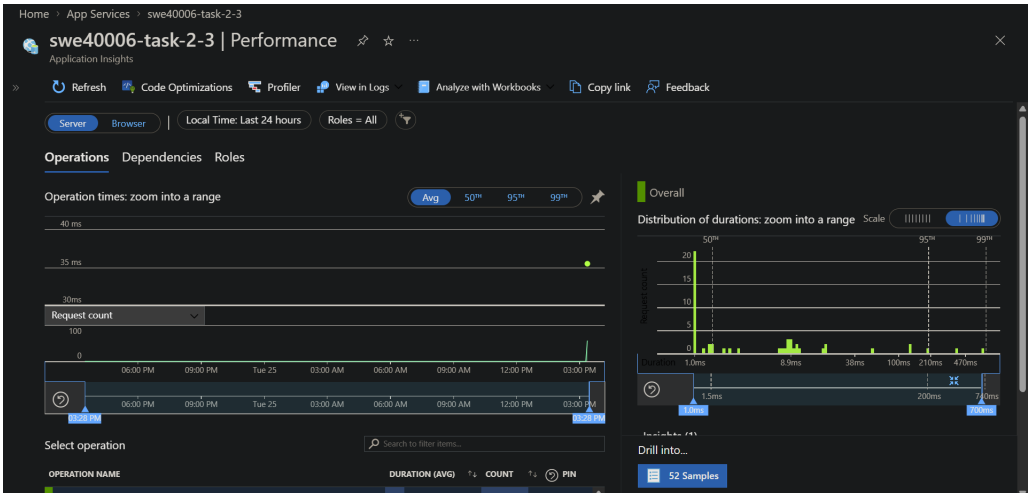


Figure 38: Performance metrics.

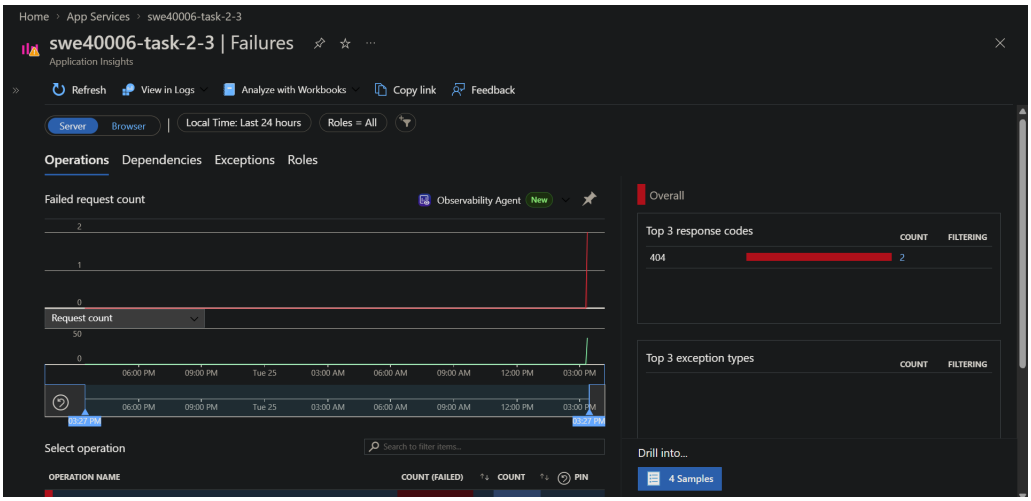


Figure 39: Failure tracking.